THE UNIVERSITY OF AZAD JAMMU & KASHMIR
DEPARTMENT OF SOFTWARE ENGINEERING
Muzaffarabad, Azad Jammu & Kashmir

LAB TASK 02

Digital Images, Pixels, Color, and Basic Transformations

Loading, inspecting, displaying, analyzing, transforming, and saving digital images with NumPy, OpenCV, and Matplotlib, covering coordinates, color channels, histograms, resolution, and bit depth.

STUDENT

Name: Shahzad Ahmed Awan

Roll No: 2024-SE-15

Semester: Fifth

Session: 2024–28 (SE)

SUBMITTED TO

Instructor: Engr. Fahad Nisar

Course: Digital Image Processing Lab

Assigned: October 22, 2026

Due: October 29, 2026

STEP 1

Project and Virtual Environment

The project folder dip-lab-02 was created, a virtual environment was built inside it, the required packages (numpy, opencv-python, and matplotlib, plus ipykernel so the notebook can run in this environment) were installed, the imports and the Python executable were verified in the terminal and in the notebook, and .venv/ was added to .gitignore.

1.1 Create the project folder

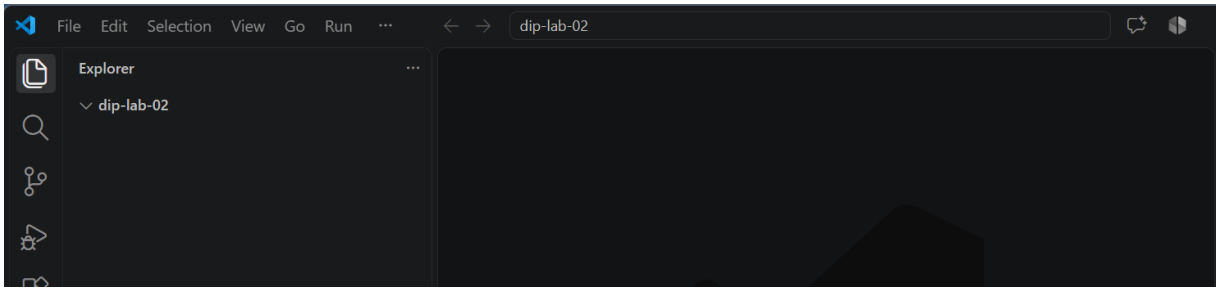


Figure 1.1 — Project folder dip-lab-02 created and opened in VS Code.

COMMANDS

```
mkdir dip-lab-02
cd dip-lab-02
```

1.2 Create and activate the virtual environment

```
PS C:\Users\ShahzadAhmedAwan\Desktop\dip-lab-02> python -m venv .venv
PS C:\Users\ShahzadAhmedAwan\Desktop\dip-lab-02>  .venv\Scripts\activate.bat
PS C:\Users\ShahzadAhmedAwan\Desktop\dip-lab-02> .venv\Scripts\Activate.ps1
>>
(.venv) PS C:\Users\ShahzadAhmedAwan\Desktop\dip-lab-02> pip install numpy opencv-python matplotlib ipykernel
```

Figure 1.2 — Virtual environment created and activated (PowerShell); the (.venv) prefix appears on the prompt.

COMMANDS

```
python -m venv .venv
.venv\Scripts\activate.bat          (PowerShell: .venv\Scripts\Activate.ps1)
```

1.3 Install the packages

```
(.venv) PS C:\Users\ShahzadAhmedAwan\Desktop\dip-lab-02> pip install numpy opencv-python matplotlib ipykernel
core, jedi, ipython-pygments-lexers, contourpy, matplotlib, jupyter-client, ipython, ipykernel
Successfully installed asttokens-3.0.2 colorama-0.4.6 comm-0.2.3 contourpy-1.4.0 cycler-0.12.1 debugpy-1.8.22 executing-2.
3.0 fonttools-4.67.0 ipykernel-7.4.0 ipython-9.17.1 ipython-pygments-lexers-1.1.1 jedi-0.20.1 jupyter-client-8.10.0 jupyte
r-core-5.9.1 kiwisolver-1.5.1 matplotlib-3.11.2 matplotlib-inline-0.2.2 nest-asyncio2-1.7.4 numpy-2.5.4 opencv-python-5.0.
0.93 packaging-26.3 parso-0.8.7 pillow-13.0.0 platformdirs-4.13.6 prompt_toolkit-3.0.53 psutil-7.2.2 pure-eval-0.2.4 pygme
nts-2.21.0 pyparsing-3.3.3 python-dateutil-3.0.0 pyzmq-27.2.0 six-1.17.0 stack_data-0.6.3 tornado-6.6 traitlets-5.16.1 typ
ing-extensions-4.16.0 tzdata-2026.5 wcwidth-0.9.4

[notice] A new release of pip is available: 26.2.1 -> 26.2.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Figure 1.3 — numpy, opencv-python, matplotlib, and ipykernel installed inside .venv.

COMMAND

```
pip install numpy opencv-python matplotlib ipykernel
```

1.4 Verify the imports and the Python executable



Figure 1.4 — Terminal check: the Python executable is inside .venv and the three libraries import correctly (OpenCV package 5.0.0.93, library version 5.0.0).

COMMAND

```
python -c "import sys, numpy, cv2, matplotlib, importlib.metadata as m; print(sys.executable); print('NumPy', m.version('numpy')); print('OpenCV', m.version('opencv-python')); print('Matplotlib', m.version('matplotlib'))"
```

1.5 Add .venv/ to .gitignore

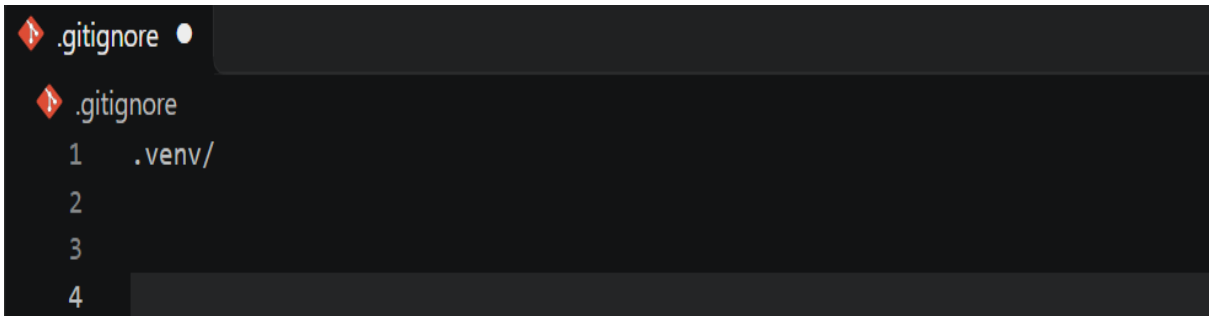


Figure 1.5 — .gitignore contains .venv/.

COMMAND

```
echo .venv/ > .gitignore
```

1.6 Verify the environment in the notebook

```
============================================================
STEP 1 — ENVIRONMENT CHECK
============================================================
Python
  Executable         : c:\Users\ShahzadAhmedAwan\Desktop\dip-lab-02\.venv\Scripts\python.exe
  Version            : 3.14.8
  Inside virtual env : True
  Working folder     : c:\Users\ShahzadAhmedAwan\Desktop\dip-lab-02

Library versions
  NumPy              : 2.5.4
  OpenCV             : 5.0.0
  Matplotlib         : 3.11.2

.gitignore
  Entries            : ['.venv/']
  .venv/ ignored     : True
```

Figure 1.6 — Notebook environment check: Python executable inside .venv, library versions, and .gitignore entries.

CODE

```
import os
import sys
import cv2 as cv
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
from IPython.display import display, Markdown


def banner(title):
    """Print a clear section header so outputs are easy to tell apart."""
    print("=" * 62)
    print(title)
    print("=" * 62)


banner("STEP 1 — ENVIRONMENT CHECK")

print("Python")
print(f"  Executable         : {sys.executable}")
print(f"  Version            : {sys.version.split()[0]}")
print(f"  Inside virtual env : {sys.prefix != sys.base_prefix}")
print(f"  Working folder     : {os.getcwd()}")

print("\nLibrary versions")
print(f"  NumPy              : {np.__version__}")
print(f"  OpenCV             : {cv.__version__}")
print(f"  Matplotlib         : {matplotlib.__version__}")

# .gitignore must exclude the virtual environment
entries = open(".gitignore").read().split() if os.path.exists(".gitignore") else []
if ".venv/" not in entries:
    with open(".gitignore", "a") as f:
        f.write(".venv/\n")
    entries.append(".venv/")

print("\n.gitignore")
print(f"  Entries            : {entries}")
print(f"  .venv/ ignored     : {'.venv/' in entries}")
```

STEP 2

Load the Image

A personally selected color image, my_image.jpeg, was placed in the project folder (not the instructor's demonstration image). It was loaded with cv.imread(); because cv.imread() returns None instead of raising an error when a file is missing, the code raises FileNotFoundError explicitly, and a second check confirms that the image is at least 300 × 300 pixels.

2.1 Input image

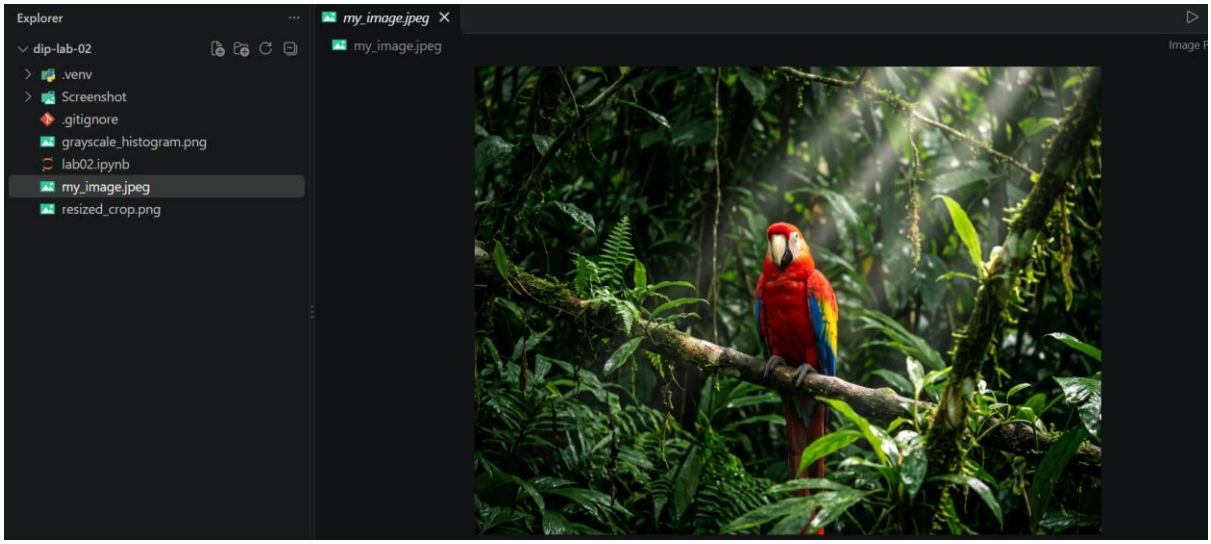


Figure 2.1 — my_image.jpeg placed in the project folder.



Figure 2.2 — The input image my_image.jpeg (2400 × 1792 pixels).

2.2 Load the image and print its shape and data type

```
============================================================
STEP 2 — IMAGE LOADED
============================================================
  File       : my_image.jpeg
  Shape      : (1792, 2400, 3)   (height, width, channels)
  Data type  : uint8
```

Figure 2.3 — Image loaded: file name, shape, and data type.

Measurement	Recorded value
File	my_image.jpeg
Image shape (height, width, channels)	(1792, 2400, 3)
Data type	uint8

CODE

```
IMAGE_PATH = "my_image.jpeg"      # <- must match your image's file name exactly

image = cv.imread(IMAGE_PATH)
if image is None:
    raise FileNotFoundError(f"Image not found or unreadable: {IMAGE_PATH}")
if min(image.shape[:2]) < 300:
    raise ValueError(f"Image must be at least 300 x 300 pixels, got {image.shape[1]} x
{image.shape[0]}")

banner("STEP 2 — IMAGE LOADED")
print(f"  File       : {IMAGE_PATH}")
print(f"  Shape      : {image.shape}   (height, width, channels)")
print(f"  Data type  : {image.dtype}")
```

STEP 3

Inspect the Image

Pixels are indexed as image[row, column], which means [y, x]. OpenCV stores the channels in Blue, Green, Red order, so every pixel is printed as (B, G, R). The code for this step is one notebook cell, shown in parts; each part is placed directly below the output it produces.

3.1 Image dimensions

```
============================================================
STEP 3 — IMAGE INSPECTION
============================================================
Dimensions
  Height          : 1792 px
  Width           : 2400 px
  Channels        : 3  (Blue, Green, Red)
  Total pixels    : 4,300,800
```

Figure 3.1 — Image dimensions (height, width, channels, and total pixels).

Measurement	Recorded value
Height	1792 px
Width	2400 px
Channels (Blue, Green, Red)	3
Total pixels	4,300,800

CODE

```
height, width, channels = image.shape
cy, cx = height // 2, width // 2          # center as [row, column] = [y, x]

top_left = image[0, 0]
center   = image[cy, cx]

banner("STEP 3 — IMAGE INSPECTION")

print("Dimensions")
print(f"  Height          : {height} px")
print(f"  Width           : {width} px")
print(f"  Channels        : {channels}  (Blue, Green, Red)")
print(f"  Total pixels    : {height * width:,}")
```

3.2 Pixel values

```
Pixel values   image[row, column]  ->  (B, G, R)
  Top-left [0, 0]       : B =   0   G =   4   R =   0
  Center [896, 1200]    : B =   0   G =   0   R = 112
```

Figure 3.2 — Top-left and center pixel values (B, G, R).

Measurement	Recorded value
Top-left pixel [0, 0] (B, G, R)	(0, 4, 0)
Center pixel [896, 1200] (B, G, R)	(0, 0, 112)

The center pixel is a dark red and lies on the parrot's chest.

CODE

```
print("\nPixel values   image[row, column]  ->  (B, G, R)")
print(f"  Top-left [0, 0]       : B = {top_left[0]:>3}   G = {top_left[1]:>3}   R =
{top_left[2]:>3}")
print(f"  Center [{cy}, {cx}]     : B = {center[0]:>3}   G = {center[1]:>3}   R = {center[2]:>3}")
```

3.3 Global and per-channel value ranges

```
Value range (0 = darkest, 255 = brightest)
  Scope            Minimum  Maximum
  ---------------------------------
  Global                 0      255
  Blue channel           0      255
  Green channel          0      255
  Red channel            0      255
```

Figure 3.3 — Global and per-channel minimum and maximum values.

Measurement	Recorded value
Global minimum / maximum	0 / 255
Blue channel minimum / maximum	0 / 255
Green channel minimum / maximum	0 / 255
Red channel minimum / maximum	0 / 255

Every channel reaches both 0 and 255, so the image uses the full 8-bit range.

CODE

```
print("\nValue range (0 = darkest, 255 = brightest)")
print(f"  {'Scope':<16}{'Minimum':>8}{'Maximum':>9}")
print(f"  {'-' * 33}")
print(f"  {'Global':<16}{image.min():>8}{image.max():>9}")
for i, name in enumerate(["Blue channel", "Green channel", "Red channel"]):
    print(f"  {name:<16}{image[:, :, i].min():>8}{image[:, :, i].max():>9}")
```

STEP 4

Convert Colors and Separate Channels

The image was converted from BGR to RGB for correct Matplotlib display, a grayscale version was created, and the red, green, and blue channels were separated. The code for this step is one notebook cell, shown in parts; each part is placed directly below the output it produces.

4.1 Six-panel color and channel figure

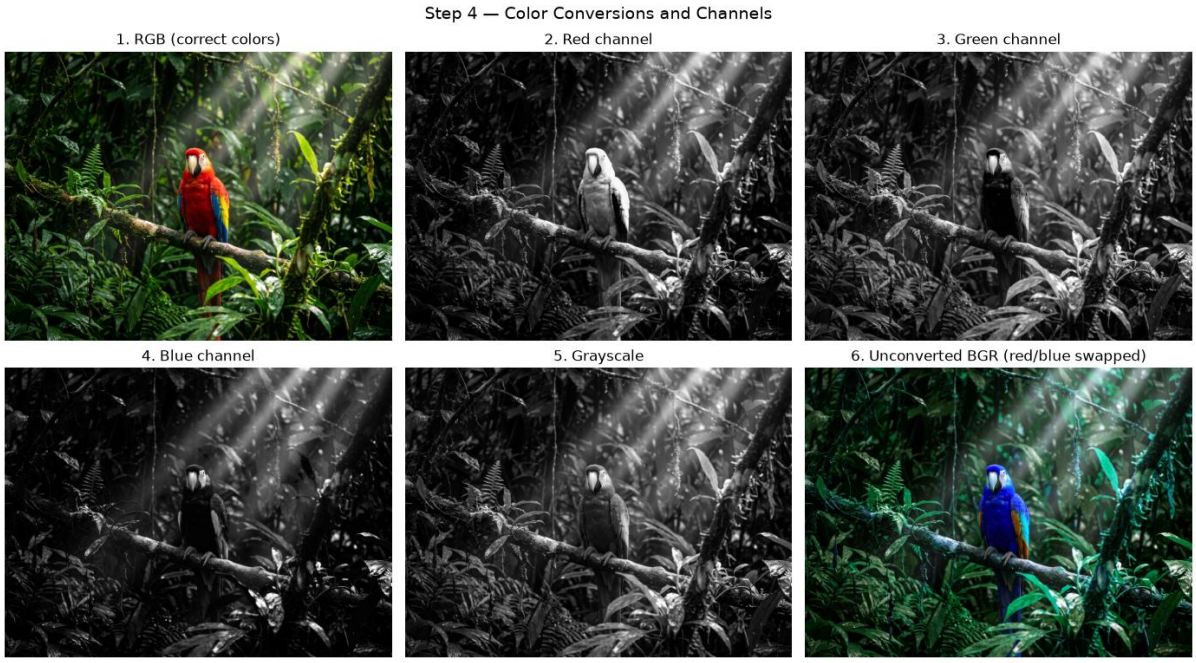


Figure 4.1 — RGB, red, green, blue, grayscale, and unconverted BGR views.

CODE

```
rgb  = cv.cvtColor(image, cv.COLOR_BGR2RGB)
gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
red, green, blue = cv.split(rgb)

views = [
    (rgb,   "1. RGB (correct colors)", None),
    (red,   "2. Red channel",          "gray"),
    (green, "3. Green channel",        "gray"),
    (blue,  "4. Blue channel",         "gray"),
    (gray,  "5. Grayscale",            "gray"),
    (image, "6. Unconverted BGR (red/blue swapped)", None),
]

fig, axes = plt.subplots(2, 3, figsize=(14, 8))
for ax, (img, title, cmap) in zip(axes.ravel(), views):
    ax.imshow(img, cmap=cmap)
    ax.set_title(title)
    ax.axis("off")
fig.suptitle("Step 4 — Color Conversions and Channels", fontsize=14)
plt.tight_layout()
plt.show()
```

4.2 Center pixel position, RGB value, and HEX value

```
============================================================
STEP 4 — CENTER PIXEL COLOR
============================================================
  Position   : [896, 1200]
  RGB        : (112, 0, 0)
  HEX        : #700000
```

Figure 4.2 — Center pixel position, RGB value, and HEX value.

Measurement	Recorded value
Center pixel position [row, column]	[896, 1200]
Center pixel RGB	(112, 0, 0)
Center pixel HEX	#700000

CODE

```
r, g, b = (int(v) for v in rgb[cy, cx])

banner("STEP 4 — CENTER PIXEL COLOR")
print(f"  Position   : [{cy}, {cx}]")
print(f"  RGB        : ({r}, {g}, {b})")
print(f"  HEX        : #{r:02X}{g:02X}{b:02X}")
```

4.3 Explanation

BGR versus RGB. RGB stores each pixel as [Red, Green, Blue], the order Matplotlib and most other libraries (such as PIL) expect. BGR stores it as [Blue, Green, Red], the order OpenCV uses when cv.imread() loads an image, kept for historical compatibility with early camera and image formats such as Windows BMP. Both hold the same three numbers; only their order is reversed (a pure red pixel is [255, 0, 0] in RGB and [0, 0, 255] in BGR).

Why the order matters. If a BGR array is shown by Matplotlib without conversion, red and blue are swapped, so the parrot's red body appears blue in panel 6, while panel 1 is correct after cv.COLOR_BGR2RGB. Greens and grays barely change, because green stays in the middle and grays have equal values. In Step 5 the image is converted back with cv.COLOR_RGB2BGR before saving, because cv.imwrite() expects BGR.

Channel views. Each channel is a single 2-D array of intensities, shown with a gray colormap: bright where that color is strong, dark where it is weak. The red channel is bright on the parrot's body, the green channel is bright on the foliage, and the blue channel is dark almost everywhere except the blue wing feathers. The grayscale image combines the three channels into one brightness value per pixel.

HEX value. The RGB value (112, 0, 0) becomes #700000, because each channel is written as a two-digit hexadecimal number (112 = 70, 0 = 00).

STEP 5

Crop, Resize, and Save

An exact 100 × 100 region centered on the image was cropped with image[y_start:y_end, x_start:x_end] and resized to 200 × 200 with cv.resize(). The original, crop, and resized crop were displayed side by side, and the resized RGB image was converted back to BGR and saved as resized_crop.png. The code for this step is one notebook cell, shown in parts; each part is placed directly below the output it produces.

5.1 Crop an exact 100 × 100 region and resize it to 200 × 200

```
============================================================
Crop region
  Rows (y)       : 846 to 946
  Columns (x)    : 1150 to 1250
  Crop shape     : (100, 100, 3)
```

Figure 5.1 — Crop region and crop shape.

Measurement	Recorded value
Rows (y)	846 to 946
Columns (x)	1150 to 1250
Crop shape	(100, 100, 3)

```
Resized
  Resized shape  : (200, 200, 3)
```

Figure 5.2 — Resized shape after cv.resize().

Measurement	Recorded value
Resized shape	(200, 200, 3)

CODE

```
y_start, y_end = cy - 50, cy + 50
x_start, x_end = cx - 50, cx + 50

crop = rgb[y_start:y_end, x_start:x_end]
assert crop.shape[:2] == (100, 100), f"Crop must be 100 x 100, got {crop.shape[:2]}"

resized = cv.resize(crop, (200, 200))      # (width, height)

banner("STEP 5 — CROP AND RESIZE")
print("Crop region")
print(f"  Rows (y)       : {y_start} to {y_end}")
print(f"  Columns (x)    : {x_start} to {x_end}")
print(f"  Crop shape     : {crop.shape}")
print("\nResized")
print(f"  Resized shape  : {resized.shape}")
```

5.2 Display the original, crop, and resized image



Figure 5.3 — Original image, crop, and resized crop side by side.

CODE

```
fig, axes = plt.subplots(1, 3, figsize=(14, 5))
panels = [
    (rgb,     f"Original ({width} x {height})"),
    (crop,    "Crop (100 x 100)"),
    (resized, "Resized (200 x 200)"),
]
for ax, (img, title) in zip(axes, panels):
    ax.imshow(img)
    ax.set_title(title)
    ax.axis("off")
fig.suptitle("Step 5 — Original, Crop, and Resized Crop", fontsize=14)
plt.tight_layout()
plt.show()
```

5.3 Save the resized crop

```
Saving
  File                : resized_crop.png
  cv.imwrite() success  : True
  Re-read shape         : (200, 200, 3)
```

Figure 5.4 — Result of cv.imwrite() and the re-read shape of resized_crop.png.

Measurement	Recorded value
File	resized_crop.png
cv.imwrite() success	True
Re-read shape	(200, 200, 3)

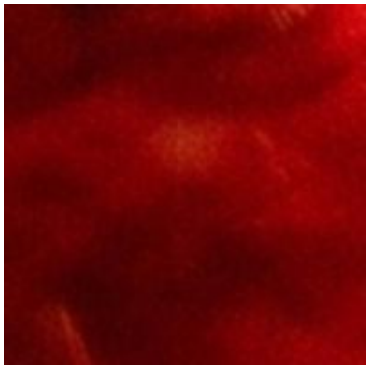


Figure 5.5 — Saved file resized_crop.png (200 × 200 pixels).

CODE

```
saved = cv.imwrite("resized_crop.png", cv.cvtColor(resized, cv.COLOR_RGB2BGR))
check = cv.imread("resized_crop.png")

print("\nSaving")
print(f"  File                : resized_crop.png")
print(f"  cv.imwrite() success  : {saved}")
print(f"  Re-read shape         : {check.shape if check is not None else 'FAILED'}")
```

STEP 6

Create a Histogram

The grayscale image was flattened and plotted as a 256-bin histogram over the range 0–256, with a title and axis labels, and saved as grayscale_histogram.png. The code for this step is one notebook cell, shown in parts; each part is placed directly below the output it produces.

6.1 Plot and save the histogram

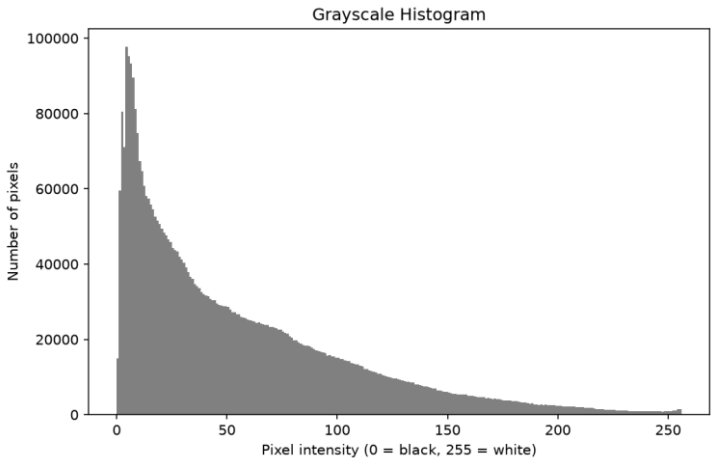


Figure 6.1 — Saved file grayscale_histogram.png.

CODE

```
plt.figure(figsize=(9, 5))
plt.hist(gray.flatten(), bins=256, range=(0, 256), color="gray")
plt.title("Grayscale Histogram")
plt.xlabel("Pixel intensity (0 = black, 255 = white)")
plt.ylabel("Number of pixels")
plt.savefig("grayscale_histogram.png", dpi=150, bbox_inches="tight")
plt.show()
```

6.2 Brightness statistics

```
============================================================
STEP 6 — HISTOGRAM STATISTICS
============================================================
  Mean intensity        : 55.4 / 255
  Dark pixels   (<85)   :  76.0 %
  Mid-tones  (85-170)   :  20.1 %
  Bright pixels (>170)  :   3.9 %
  Plot saved as         : grayscale_histogram.png
```

Figure 6.2 — Histogram statistics: mean intensity and dark, mid, and bright pixel percentages.

Measurement	Recorded value
Mean intensity	55.4 / 255
Dark pixels (below 85)	76.0%
Mid pixels (85 to 170)	20.1%
Bright pixels (above 170)	3.9%
Plot saved as	grayscale_histogram.png

CODE

```
mean   = gray.mean()
dark   = (gray < 85).mean() * 100
mid    = ((gray >= 85) & (gray <= 170)).mean() * 100
bright = (gray > 170).mean() * 100

banner("STEP 6 — HISTOGRAM STATISTICS")
print(f"  Mean intensity        : {mean:.1f} / 255")
print(f"  Dark pixels   (<85)   : {dark:5.1f} %")
print(f"  Mid-tones  (85-170)   : {mid:5.1f} %")
print(f"  Bright pixels (>170)  : {bright:5.1f} %")
print(f"  Plot saved as         : grayscale_histogram.png")
```

6.3 Interpretation

The grayscale histogram has a mean intensity of 55.4 out of 255, with 76.0% of pixels in the dark range and 3.9% in the bright range. The image is therefore mainly dark. Most pixels sit at the low-intensity end, and only a thin tail reaches the bright end.

CODE

```
# Interpretation generated from the measured values above
if mean < 85:
    verdict = "mainly dark"
    detail = "Most pixels sit at the low-intensity end, and only a thin tail reaches the bright end."
elif mean > 170:
    verdict = "mainly bright"
    detail = "Most pixels sit at the high-intensity end, and only a thin tail reaches the dark end."
else:
    verdict = "balanced"
    detail = "Pixels are spread across the dark, mid, and bright ranges without one end dominating."

display(Markdown(
    f"### Interpretation\n\n"
    f"The grayscale histogram has a mean intensity of **{mean:.1f} out of 255**, with "
    f"**{dark:.1f}%** of pixels in the dark range and **{bright:.1f}%** in the bright range. "
    f"The image is therefore **{verdict}**. {detail}"
))
```

What a histogram shows and does not show. A histogram counts how many pixels fall into each of the 256 intensity values, so it reveals overall brightness, contrast, clipping at pure black or white, and dominant tones. It does not show where the pixels are (shuffling them would leave the histogram unchanged), what the image contains (unrelated photographs can share a histogram), color (a grayscale histogram merges the three channels), structure such as edges and texture, or whether a dark image comes from a dark scene or from underexposure.

STEP 7

Demonstrate Bit Depth

An 8-bit horizontal gradient from 0 to 255 was created and quantized to 1-bit (2 levels), 2-bit (4 levels), and 4-bit (16 levels). The code for this step is one notebook cell, shown in parts; each part is placed directly below the output it produces.

7.1 Bit-depth comparison figure

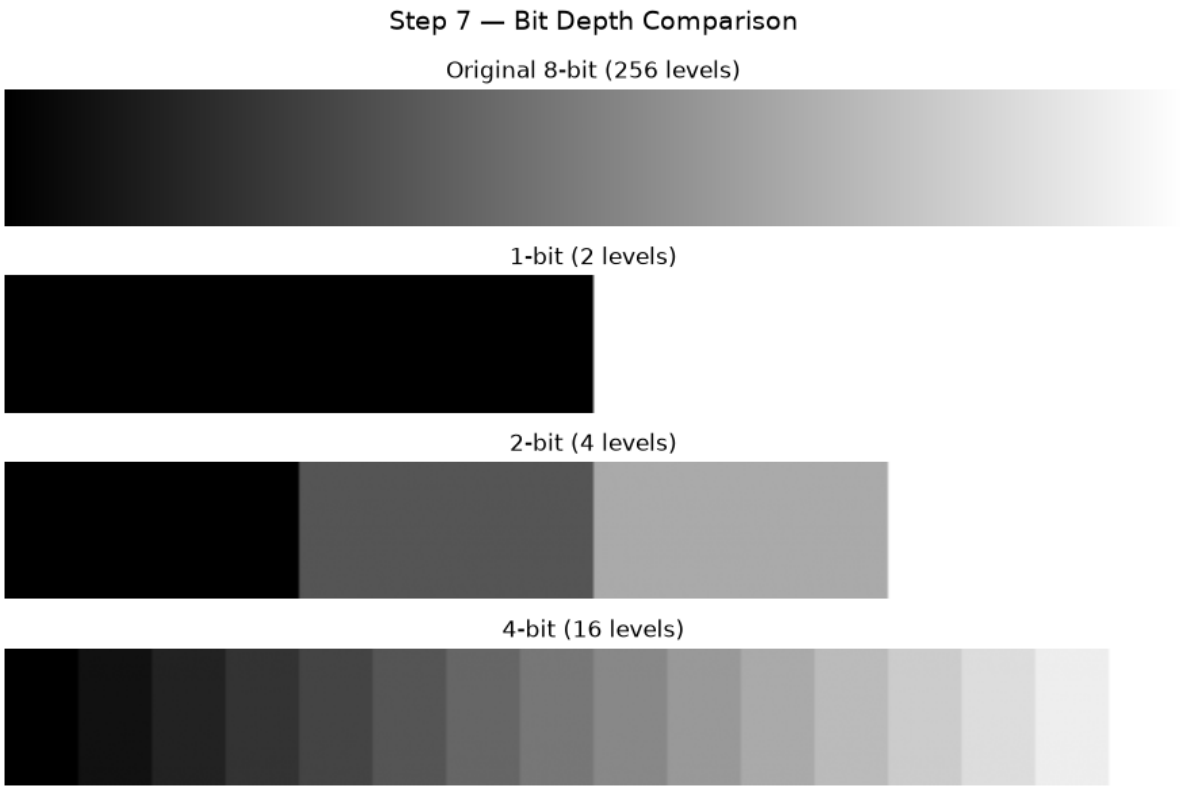


Figure 7.1 — Original 8-bit gradient and the 1-, 2-, and 4-bit versions.

CODE

```
gradient = np.tile(np.arange(256, dtype=np.uint8), (100, 1))      # 8-bit horizontal ramp, 0 to 255


def quantize(img, bits):
    levels = 2 ** bits
    return ((img // (256 // levels)) * (255 // (levels - 1))).astype(np.uint8)


panels = [
    (gradient,              "Original 8-bit (256 levels)", 8),
    (quantize(gradient, 1), "1-bit (2 levels)",            1),
    (quantize(gradient, 2), "2-bit (4 levels)",            2),
    (quantize(gradient, 4), "4-bit (16 levels)",           4),
]

fig, axes = plt.subplots(4, 1, figsize=(9, 6))
for ax, (img, title, bits) in zip(axes, panels):
    ax.imshow(img, cmap="gray", vmin=0, vmax=255, aspect="auto")
    ax.set_title(title)
    ax.axis("off")
fig.suptitle("Step 7 — Bit Depth Comparison", fontsize=14)
plt.tight_layout()
plt.show()
```

7.2 Unique-value counts

```
================================================================
STEP 7 — UNIQUE VALUE COUNTS
================================================================
  Image                          Unique   Expected   Result
  -------------------------------------------------------
  Original 8-bit (256 levels)       256        256       OK
  1-bit (2 levels)                    2          2       OK
  2-bit (4 levels)                    4          4       OK
  4-bit (16 levels)                  16         16       OK
```

Figure 7.2 — Unique-value counts compared with the expected counts.

Image	Unique values (expected)
Original 8-bit (256 levels)	256 (expected 256, OK)
1-bit (2 levels)	2 (expected 2, OK)
2-bit (4 levels)	4 (expected 4, OK)
4-bit (16 levels)	16 (expected 16, OK)

CODE

```
banner("STEP 7 — UNIQUE VALUE COUNTS")
print(f"  {'Image':<28}{'Unique':>8}{'Expected':>10}{'Result':>9}")
print(f"  {'-' * 55}")
for img, title, bits in panels:
    unique, expected = len(np.unique(img)), 2 ** bits
    print(f"  {title:<28}{unique:>8}{expected:>10}{'OK' if unique == expected else 'FAIL':>9}")
```

7.3 Explanation

Resolution versus bit depth. Resolution is the number of pixels, width × height, and controls spatial detail; lowering it leaves fewer samples, so edges turn into staircases and the image looks blocky. Bit depth is the number of bits used for each pixel value, which sets the number of distinct intensities (2^n: 2 levels for 1-bit, 4 for 2-bit, 16 for 4-bit, 256 for 8-bit) and controls tonal precision; lowering it causes banding, where a smooth gradient turns into visible steps. In this lab, the 100 × 100 crop and the 200 × 200 resized version differ in resolution, while in Step 7 the gradient keeps all 256 columns (resolution unchanged) yet shows only 2, 4, or 16 shades as the bit depth drops.

They are independent. An image can have high resolution and low bit depth (many pixels, few shades) or the reverse. Both affect file size, which is roughly width × height × bits per pixel ÷ 8 bytes. A 640 × 480 grayscale image at 8 bits needs about 307,200 bytes; reducing it to 2 bits gives 76,800 bytes, and halving the resolution to 320 × 240 at 8 bits also gives 76,800 bytes, yet the first result looks banded while the second looks blocky. In short, resolution answers “how many samples are there?”, and bit depth answers “how precise is each sample?”.

STEP 8

Final Run and Submission

The notebook lab02.ipynb was run from the top with Run All inside the activated .venv environment, all errors were corrected, the required files were checked, and deactivate was run when finished.

8.1 Required-file check

```
============================================================
STEP 8 — FINAL CHECK
============================================================
  File                       Status          Size
  -------------------------------------------------
  my_image.jpeg              found       3381.0 KB
  resized_crop.png           found         37.6 KB
  grayscale_histogram.png    found         27.3 KB
  .gitignore                 found          0.0 KB

All cells completed successfully.
```

Figure 8.1 — Required files found, with their sizes.

CODE

```
banner("STEP 8 — FINAL CHECK")
print(f"  {'File':<28}{'Status':<10}{'Size':>10}")
print(f"  {'-' * 48}")
for name in [IMAGE_PATH, "resized_crop.png", "grayscale_histogram.png", ".gitignore"]:
    if os.path.exists(name):
        print(f"  {name:<28}{'found':<10}{os.path.getsize(name) / 1024:>8.1f} KB")
    else:
        print(f"  {name:<28}{'MISSING':<10}")
print("\nAll cells completed successfully.")
```

8.2 Project folder contents

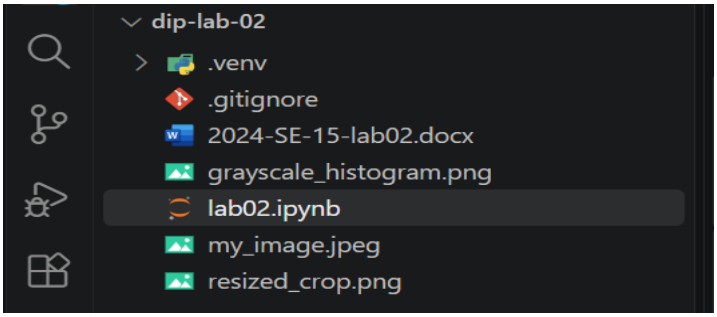


Figure 8.2 — Project folder with the files to submit.

8.3 Deactivate the environment

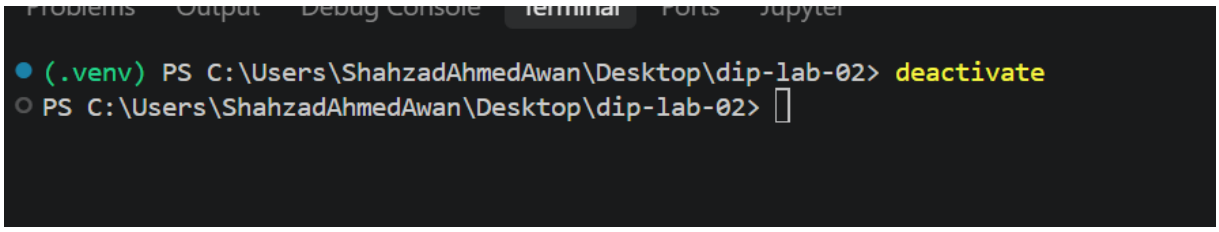


Figure 8.3 — Virtual environment deactivated.

COMMAND

```
deactivate
```

8.4 Submitted files

File	Description
lab02.ipynb	Notebook with the complete code and outputs
my_image.jpeg	Input image
resized_crop.png	200 × 200 resized crop
grayscale_histogram.png	Saved grayscale histogram
.gitignore	Contains .venv/
2024-SE-15-lab02.docx	This report

The .venv folder is not submitted.